



Stack, Queue & Monotonic Deque

Nội dung

- 1. Stack & Queue cơ bản
- 2. Bài toán mở đầu: Dấu ngoặc hợp lệ
- 3. Monotonic Stack — Stack đơn điệu
 - Bài toán: Nearest Smaller Value (giá trị nhỏ hơn gần nhất bên trái)
 - Hình dung — chạy tay với $(a = [3, 1, 4, 1, 5])$
 - Khi nào dùng monotonic stack?
- 4. Monotonic Deque — cho cửa sổ trượt
 - Hình dung — chạy tay với $(a = [3, 1, 5, 2, 4])$, $(k = 3)$
 - Cốt lõi của ý tưởng
- 5. Histogram & Maximum Rectangle (mẫu khó hơn)
 - Cài đặt (gói trong một hàm)
 - Mở rộng: Hình chữ nhật lớn nhất toàn 1 trong ma trận nhị phân
- 6. Tổng kết & Mẹo

Stack và Queue là hai cấu trúc dữ liệu **siêu cơ bản** — nhưng kết hợp với ý tưởng **đơn điệu (monotonic)** thì chúng trở thành công cụ giải hàng loạt bài toán "nhìn có vẻ" cần segment tree.

1. Stack & Queue cơ bản

- **Stack** (LIFO — Last In First Out): `push`, `pop`, `top` đều $O(1)$.
- **Queue** (FIFO — First In First Out): `push`, `pop`, `front` đều $O(1)$.
- **Deque** (Double-Ended Queue): thêm/xóa cả hai đầu trong $O(1)$.

C++

Python



Tiếng Việt



```
#include <queue>
#include <deque>

stack<int> st;    st.push(5); st.top(); st.pop();
queue<int> q;      q.push(5); q.front(); q.pop();
deque<int> dq;     dq.push_back(5); dq.push_front(3);
                  dq.pop_back(); dq.pop_front();
```

```
from collections import deque

# Stack: dùng list, append / pop ở cuối – đều O(1)
st = []
st.append(5); st[-1]; st.pop()

# Queue: dùng deque, KHÔNG dùng list (list.pop(0) là O(n)!)
q = deque()
q.append(5); q[0]; q.popleft()

# Deque: thêm/xóa cả hai đầu trong O(1)
dq = deque()
dq.append(5); dq.appendleft(3)
dq.pop(); dq.popleft()
```

“

Bẫy Python: KHÔNG dùng `List` làm queue — `List.pop(0)` là $O(n)$ vì phải dịch chuyển toàn bộ phần tử. Luôn dùng `collections.deque` cho queue và deque.

2. Bài toán mở đầu: Dấu ngoặc hợp lệ

Đề: Cho xâu gồm `()[]{}.` Kiểm tra xem có hợp lệ không.

Ý tưởng: Mở thì push, đóng thì pop và so sánh.

C++

Python



```

for (char c : s) {
    if (c == '(' || c == '[' || c == '{') st.push(c);
    else {
        if (st.empty()) return false;
        char x = st.top(); st.pop();
        if ((c == ')' && x != '(') ||
            (c == ']' && x != '[') ||
            (c == '}' && x != '{')) return false;
    }
}
return st.empty();

```

```

pair = {')': '(', ']': '[', '}': '{'}
st = []
for c in s:
    if c in '([{':
        st.append(c)
    else:
        if not st or st.pop() != pair[c]:
            return False
return not st

```

Đây là **mẫu kinh điển** — nhớ thật kỹ!

3. Monotonic Stack — Stack đơn điệu

Ý tưởng: Duy trì một stack mà các phần tử bên trong **luôn tăng** (hoặc **luôn giảm**) từ đáy lên đỉnh.

Bài toán: Nearest Smaller Value (giá trị nhỏ hơn gần nhất bên trái)

Đề: Cho dãy $a_1, a_2, \dots, a_n$. Với mỗi i , tìm j lớn nhất với $j < i$ và $a_j < a_i$. Nếu không có, in 0.

Cách $O(n^2)$: Với mỗi i , duyệt ngược j — quá chậm.

Cách $O(n)$ với monotonic stack: Duy trì stack lưu các index j với a_j **tăng dần**. Khi xét i mới:



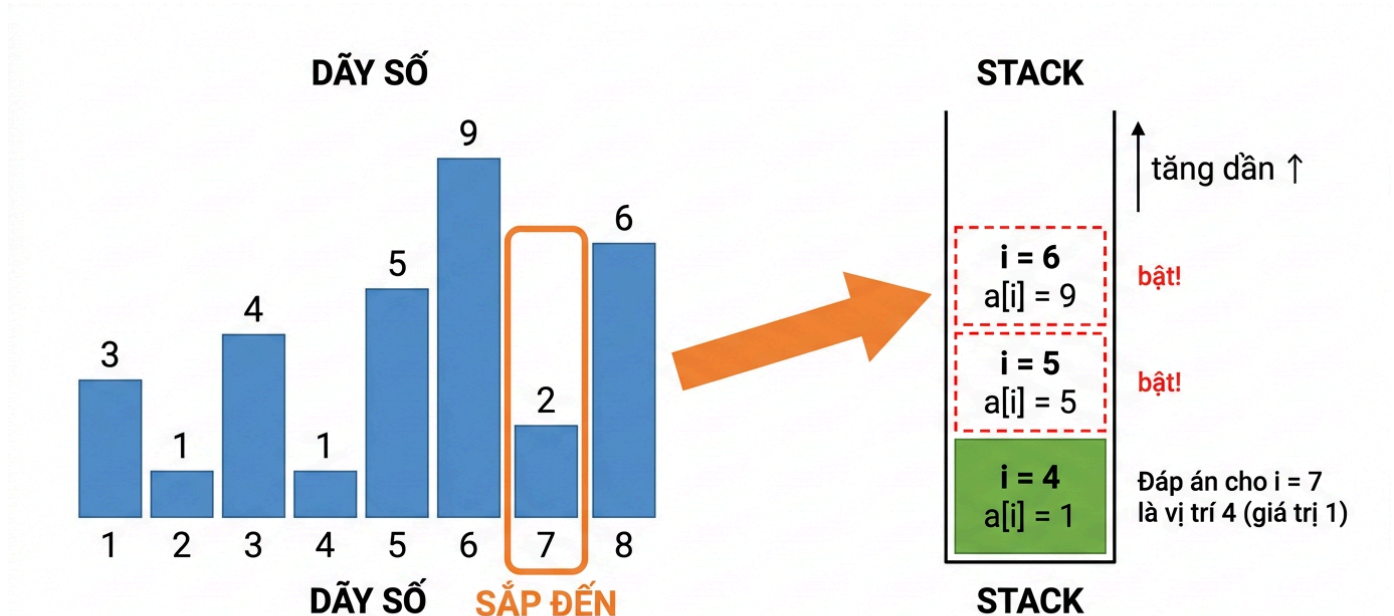
monj.



Tiếng Việt



- Đỉnh stack hiện tại (nếu có) chính là đáp án.
- Push i vào stack.



Stack luôn giữ giá trị tăng dần (đáy $\rightarrow$ đỉnh). Phần tử mới bật mọi giá trị $\geq$ **chính nó**.

Hình trên minh họa **bất biến** của monotonic stack với dãy $a = [3, 1, 4, 1, 5, 9, 2, 6]$, ngay trước khi xét $i = 7$ (giá trị 2). Ở mọi thời điểm, các giá trị trong stack đều **tăng dần từ đáy lên đỉnh** — lúc này stack chứa $\{i = 4, i = 5, i = 6\}$ với giá trị $\{1, 5, 9\}$ (các vị trí 2, 3 đã bị bật từ trước, lúc $i = 4$). Khi $a_7 = 2$ tới, ta bật tất cả giá trị ≥ 2 — đó là $i = 6$ (giá trị 9) và $i = 5$ (giá trị 5). $i = 4$ giữ lại vì $1 < 2$. Đỉnh stack sau khi bật chính là **đáp án "nhỏ hơn gần nhất bên trái"** cho $i = 7$, tức là $i = 4$.

Hình dung — chạy tay với $a = [3, 1, 4, 1, 5]$

Stack chứa **chỉ số** sao cho giá trị tương ứng **tăng dần từ đáy $\rightarrow$ đỉnh**.



Tiếng Việt



stack val: [3]

i=2, a[i]=1: a[1]=3 $\geq$ 1 $\rightarrow$ pop 1
 stack rỗng $\rightarrow$ ans[2] = 0
 push 2

stack idx: []

stack idx: [2]

stack val: [1]

i=3, a[i]=4: a[2]=1 < 4 $\rightarrow$ KHÔNG pop
 top = 2 $\rightarrow$ ans[3] = 2
 push 3

stack idx: [2, 3]

stack val: [1, 4]

i=4, a[i]=1: a[3]=4 $\geq$ 1 $\rightarrow$ pop 3
 a[2]=1 $\geq$ 1 $\rightarrow$ pop 2
 stack rỗng $\rightarrow$ ans[4] = 0
 push 4

stack idx: []

stack idx: [4]

stack val: [1]

i=5, a[i]=5: a[4]=1 < 5 $\rightarrow$ KHÔNG pop
 top = 4 $\rightarrow$ ans[5] = 4
 push 5

stack idx: [4, 5]

stack val: [1, 5]

Đáp án: `ans = [0, 0, 2, 0, 4]`. Mỗi chỉ số được push 1 lần, pop tối đa 1 lần $\Rightarrow$ tổng $O(n)$ amortized.

C++

Python

```
stack<int> st;
for (int i = 1; i <= n; i++) {
    while (!st.empty() && a[st.top()] >= a[i]) st.pop();
    ans[i] = st.empty() ? 0 : st.top();
    st.push(i);
}
```



Tiếng Việt



```
ans = [0] * (n + 1)
for i in range(1, n + 1):
    while st and a[st[-1]] >= a[i]:
        st.pop()
    ans[i] = st[-1] if st else 0
    st.append(i)
```

Tại sao $O(n)$? Mỗi phần tử được push 1 lần và pop tối đa 1 lần — **amortized $O(n)$** .

Khi nào dùng monotonic stack?

- Tìm "phần tử gần nhất bên trái/phải nhỏ hơn / lớn hơn"
- Tính diện tích lớn nhất hình chữ nhật trong histogram
- Dạng "với mỗi i , tìm khoảng $[l, r]$ mà a_i là min/max"

4. Monotonic Deque — cho cửa sổ trượt

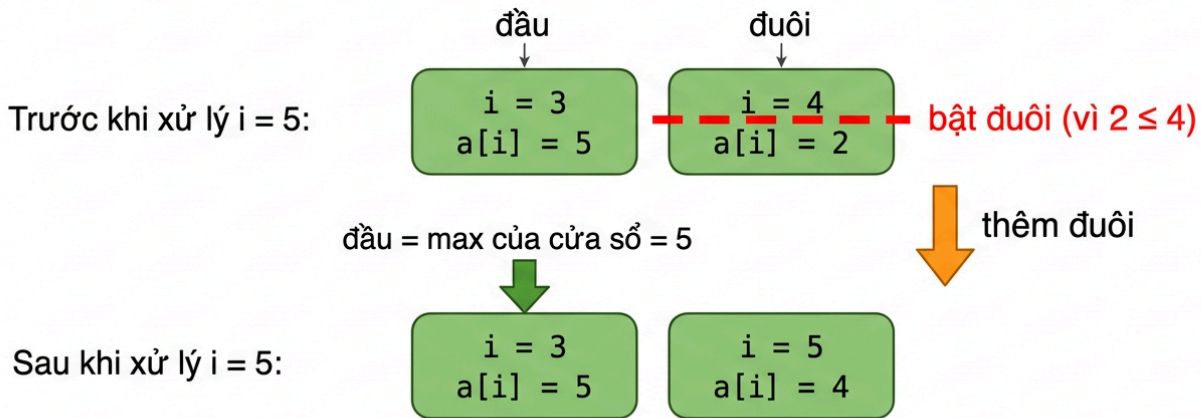
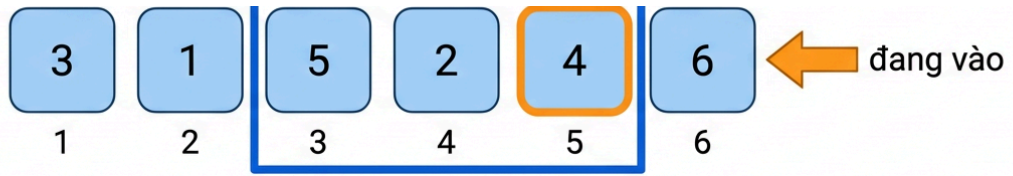
Bài toán: Cho dãy $a_1, \dots, a_n$ và k . Với mỗi cửa sổ độ dài k liên tiếp, in giá trị **lớn nhất**.

Cách ngây thơ: $O(nk)$ — chậm.

Cách $O(n)$ với deque: Duy trì deque chứa **index** với giá trị a **giảm dần** từ đầu đến cuối deque.

Khi trượt sang phải tới i :

1. **Xóa từ đuôi** các phần tử có a nhỏ hơn a_i (chúng không bao giờ là max trong cửa sổ chứa i).
2. **Push i vào đuôi**.
3. **Xóa từ đầu** nếu $\text{index} \leq i - k$ (đã ngoài cửa sổ).
4. Đầu deque là index của max.



Bất biến: giá trị trong deque **GIẢM DẦN** từ đầu xuống đuôi. Đầu deque luôn = max của cửa sổ.

Hình trên minh họa **một bước** xử lý trên dãy $a = [3, 1, 5, 2, 4, 6]$ với $k = 3$, ngay khi $i = 5$ (giá trị 4) chuẩn bị vào cửa sổ. Trước bước này, deque chứa $\{i = 3, i = 4\}$ với giá trị $\{5, 2\}$ — giảm dần từ đầu xuống đuôi. Khi $a_5 = 4$ tới:

- **Bật đuôi:** $a[i = 4] = 2 \leq 4$, nên $i = 4$ là **vô dụng** (vừa bé hơn 4 vừa "chết" sớm hơn) → bỏ.
- **Thêm đuôi:** push $i = 5$.

Sau bước, deque = $\{i = 3, i = 5\}$ với giá trị $\{5, 4\}$ — vẫn giảm dần. **Đầu deque** = $i = 3$ chính là vị trí của max trong cửa sổ $[3..5]$ (giá trị 5).

Hình dung — chạy tay với $a = [3, 1, 5, 2, 4], k = 3$

Deque luôn chứa **chỉ số** sao cho giá trị tương ứng **giảm dần từ đầu → cuối**.



Pop đầu (rỗng): skip

Push 1 → deque (idx): [1]
→ giá trị: [3]

i=2, a[i]=1

a[1]=3 > a[2]=1 → KHÔNG pop

Push 2 → deque (idx): [1, 2]
→ giá trị: [3, 1]

i=3, a[i]=5

a[2]=1 ≤ 5 → pop 2 → [1]

a[1]=3 ≤ 5 → pop 1 → []

Push 3 → deque (idx): [3]
→ giá trị: [5]

Cửa sổ [1..3]: max = a[deque.front()] = a[3] = 5 ✓

i=4, a[i]=2

a[3]=5 > 2 → KHÔNG pop

Push 4 → deque (idx): [3, 4]
→ giá trị: [5, 2]

Front = 3 ≥ i-k+1 = 2 → trong cửa sổ

Cửa sổ [2..4]: max = a[3] = 5 ✓

i=5, a[i]=4

a[4]=2 ≤ 4 → pop 4 → [3]

a[3]=5 > 4 → KHÔNG pop

Push 5 → deque (idx): [3, 5]
→ giá trị: [5, 4]

Front = 3 < i-k+1 = 3? Không (3 = 3) → vẫn trong cửa sổ

Cửa sổ [3..5]: max = a[3] = 5 ✓

Quan sát: **giá trị trong deque luôn giảm dần** ⇒ front = max. Mỗi chỉ số chỉ bị push 1 lần và pop tối đa 1 lần ⇒ tổng $O(n)$.

C++

Python



```
for (int i = 1; i <= n; i++) {
    while (!dq.empty() && a[dq.back()] <= a[i]) dq.pop_back();
    dq.push_back(i);
    if (dq.front() <= i - k) dq.pop_front();
    if (i >= k) cout << a[dq.front()] << ' ';
}
```

```
from collections import deque
dq = deque()
out = []
for i in range(n):
    while dq and a[dq[-1]] <= a[i]:
        dq.pop()
    dq.append(i)
    if dq[0] <= i - k:
        dq.popleft()
    if i >= k - 1:
        out.append(a[dq[0]])
print(*out)
```

Đổi thành để tìm min.

Cốt lõi của ý tưởng

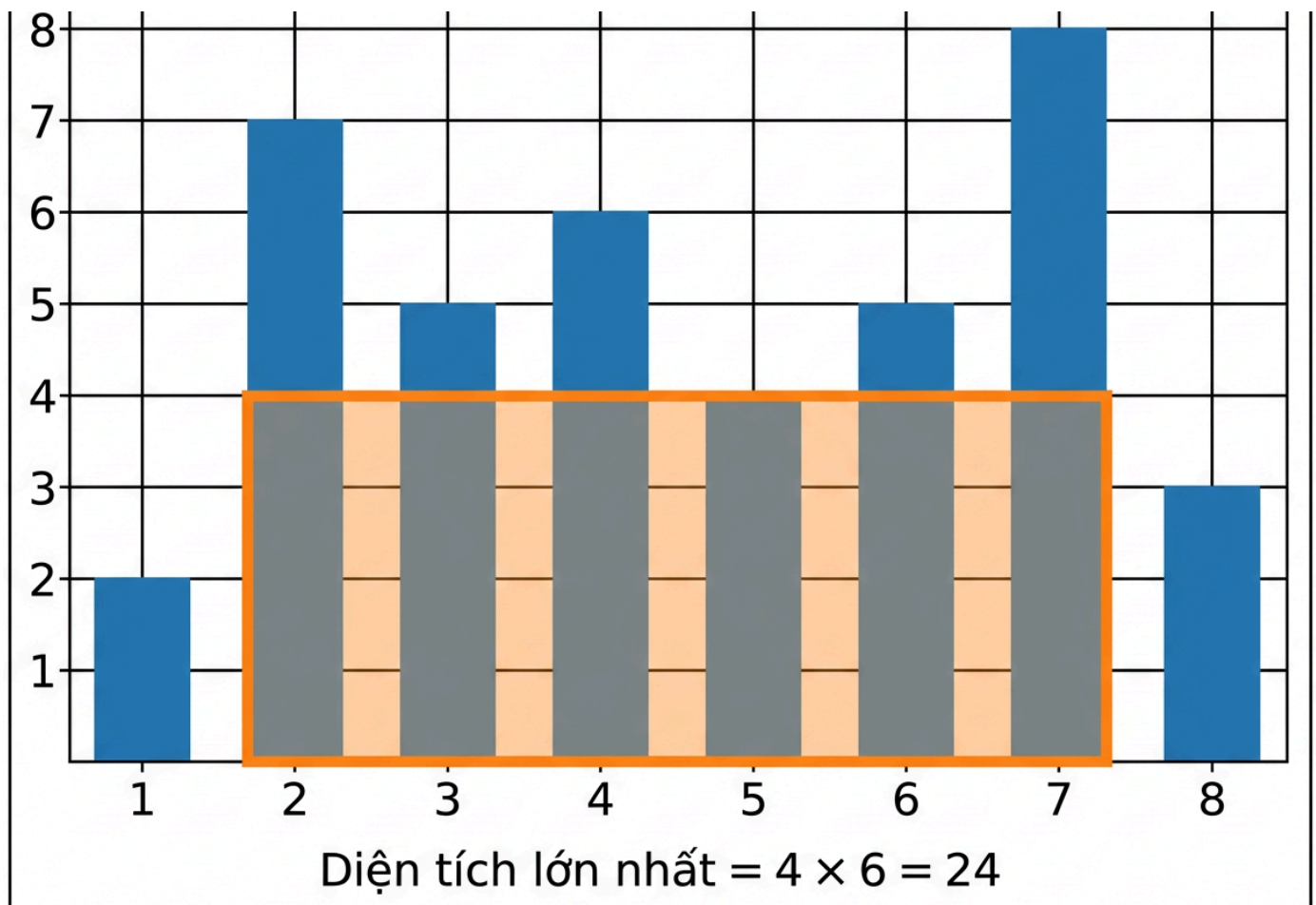
Khi a_i vào cửa sổ, mọi phần tử **nhỏ hơn a_i và đứng trước** đều **vô dụng** mãi mãi (vì a_i vừa lớn hơn vừa "sống lâu" hơn trong cửa sổ). Vậy ta xóa chúng.

5. Histogram & Maximum Rectangle (mẫu khó hơn)

Đề: Cho histogram với cột cao $h_1, \dots, h_n$ (rộng 1). Tìm hình chữ nhật có diện tích lớn nhất nằm gọn trong histogram.



Tiếng Việt



Trong ví dụ trên, dãy chiều cao là $[2, 7, 5, 6, 4, 5, 8, 3]$. Hình chữ nhật lớn nhất (vùng cam) **trải dài từ cột 2 đến cột 7** với chiều cao **bị giới hạn bởi cột 5 (cao nhất là 4)**, cho diện tích $4 \times 6 = 24$. Lưu ý: hai cột "thấp hơn gần nhất" — cột 1 (cao 2) bên trái và cột 8 (cao 3) bên phải — đều cách trung tâm hình chữ nhật **vài cột**, nên thuật toán phải "nhìn lùi xa" để biết được biên đúng.

Ý tưởng: Với mỗi cột i , tìm L_i = vị trí cột thấp hơn gần nhất bên trái, R_i = bên phải. Khi đó, hình chữ nhật cao h_i rộng nhất là từ $L_i + 1$ tới $R_i - 1$, diện tích = $h_i \cdot (R_i - L_i - 1)$.

Cả L và R đều tính bằng monotonic stack trong $O(n)$ — đây là **bài toán quan trọng** đáng nhớ.

Cài đặt (gói trong một hàm)

Mẹo cài đặt gọn: thêm hai cột "ảo" cao -1 ở đầu và cuối. Khi đó mọi cột thực sẽ có thấp hơn ở cả hai phía → không cần xử lý biên đặc biệt.

C++

Python



Tiếng Việt



```

int n = h.size();
h.push_back(-1); // Cột "ảo" thấp hơn mọi cột → buộc
pop hết
stack<int> st;
long long ans = 0;
for (int i = 0; i <= n; i++) {
    while (!st.empty() && h[st.top()] >= h[i]) {
        int j = st.top(); st.pop();
        int left = st.empty() ? -1 : st.top();
        ans = max(ans, (long long)h[j] * (i - left - 1));
    }
    st.push(i);
}
return ans;
}

```

```

def largest_rectangle_in_histogram(h):
    h = h + [-1] # Cột "ảo" thấp hơn mọi cột
    st = []
    ans = 0
    for i, x in enumerate(h):
        while st and h[st[-1]] >= x:
            j = st.pop()
            left = st[-1] if st else -1
            ans = max(ans, h[j] * (i - left - 1))
        st.append(i)
    return ans

```

Cách hoạt động: khi pop cột j vì cột i "thấp hơn (hoặc bằng)", ta biết i chính là biên phải R_j và đỉnh stack mới là biên trái L_j . Hình chữ nhật cao $h[j]$ rộng từ $L_j + 1$ tới $R_j - 1$.

Mở rộng: Hình chữ nhật lớn nhất toàn 1 trong ma trận nhị phân

Đề: Cho ma trận $m \times n$ gồm 0 và 1. Tìm hình chữ nhật con lớn nhất chỉ chứa toàn 1.

Ý tưởng — quy về histogram trên từng dòng:



Tiếng Việt



xuong). Cập nhật.

2. $h_i[j] = h_{i-1}[j] + 1$ nếu $a[i][j] = 1$
3. $h_i[j] = 0$ nếu $a[i][j] = 0$
4. Sau đó dòng i trở thành một **histogram** với các cột cao $h_i[1], h_i[2], \dots, h_i[n]$. Hình chữ nhật toàn 1 lớn nhất **kết thúc ở dòng i** chính là max rectangle trong histogram đó.
5. Đáp án = max qua mọi dòng.

Phức tạp: $O(mn)$ — mỗi dòng dùng histogram max rect trong $O(n)$.

C++

Python

```
int h[N] = {0}, ans = 0;
for (int i = 1; i <= m; i++) {
    for (int j = 1; j <= n; j++) {
        h[j] = (a[i][j] == 1) ? h[j] + 1 : 0;
    }
    ans = max(ans, largestRectangleInHistogram(vector<int>(h + 1, h + n + 1)));
}
```

```
h = [0] * (n + 1)
ans = 0
for i in range(1, m + 1):
    for j in range(1, n + 1):
        h[j] = h[j] + 1 if a[i][j] == 1 else 0
    ans = max(ans, largest_rectangle_in_histogram(h))
```

Đây là **kỹ thuật giảm bài toán 2D về 1D** — một mẫu rất phổ biến trong CP. Hãy nhớ kỹ!

6. Tổng kết & Mẹo

- **Stack** giải bài "ngoặc", "histogram", "nearest smaller/greater".
- **Queue** dùng cho BFS (sẽ thấy trong bài 9).
- **Monotonic Deque** giải sliding window min/max trong $O(n)$.
- Thao tác cốt lõi: "phần tử mới đến → xóa hết phần tử ở đuôi mà 'vô dụng' → push".



Tiếng Việt



- Khi gặp "max/min trong mọi cửa sổ kích thước k ", hãy nghĩ **monotonic deque**.

Luyện tập là chìa khóa — làm 5-7 bài bạn sẽ nhận ra pattern ngay!

Bài tập

Biểu thức hậu tố	0 / 10
Xếp hàng	0 / 15
Thực hiện biểu thức	0 / 15
Xâu Ami	0 / 15
Xây dựng mảng	0 / 20
CSES - Nearest Smaller Values Giá trị nhỏ hơn gần nhất	0 / 25
Đủ chất	0 / 25
Giá trị nhỏ nhất	0 / 25
Trạm xăng	0 / 30
HIST	0 / 25
CSES - Advertisement Quảng cáo	0 / 30
Hình chữ nhật lớn nhất	0 / 30
Hình chữ nhật 0 1	0 / 30

proudly powered by **DMOJ** | [developed by LQDJudge team](#) Tiếng Việt (vi) ▼